Doe Language Documentation

Current Version: `Dough_V0.8.0-alpha`

Aidanace3

Contents

1 Getting Started **3**
- 1.1 Running Doe Programs . . . 3
- 1.2 VS Code Extension . . . 3
- 1.3 Install Runtime (CLI) . . . 3
- 1.4 VS Code `launch.json` . . . 3

2 Section 1: Basic Syntax **4**
- 2.1 General Rules . . . 4
- 2.2 Operators . . . 4
  - 2.2.1 Arithmetic Operators . . . 4
  - 2.2.2 Comparison & Logic Operators . . . 4
- 2.3 Types . . . 4
- 2.4 I/O . . . 5
  - 2.4.1 Output . . . 5
  - 2.4.2 Input . . . 5
- 2.5 Conditions . . . 5
  - 2.5.1 If / Elif / Else . . . 5
  - 2.5.2 Calling a Function on Condition . . . 5
  - 2.5.3 Switch (`IfCase`) . . . 6
- 2.6 Functions . . . 6
- 2.7 Points . . . 6

3 Section 2: Examples & Syntax **8**
- 3.1 Loops . . . 8
  - 3.1.1 Point Loop . . . 8
  - 3.1.2 While Loop (`as`) . . . 8
  - 3.1.3 For-Each Loop . . . 8
  - 3.1.4 Counted For Loop . . . 8
- 3.2 Arrays . . . 8
  - 3.2.1 Array Operations . . . 9
- 3.3 Dictionaries . . . 9
  - 3.3.1 Standard Dictionary . . . 9
  - 3.3.2 Locked (Single-Type) Dictionary . . . 9
- 3.4 Points — Full Examples . . . 9
  - 3.4.1 Basic Point (One-Shot) . . . 9
  - 3.4.2 Continuous Listener . . . 10
  - 3.4.3 Storing & Requesting Values via a Point . . . 10
- 3.5 Error Handling . . . 10
- 3.6 Pattern Matching (`match`) . . . 11

3.7 Async Points & Concurrency 11
3.8 Imports & Modules 11

4 Section 3: Quick Reference **12**
4.1 Keyword Cheatsheet 12
4.2 Common Patterns 12
4.2.1 Safe Division 12
4.2.2 Collecting Input into an Array 12
4.2.3 Event Bus with Points 13

Getting Started

Install the runtime: https://github.com/Aidanace3/Dough

Running Doe Programs

Command	Description
`.\dough.cmd examples\test.doe`	Direct run (default verbose)
`.\dough.ps1 examples\test.doe`	PowerShell wrapper
`.\dough.cmd --silent examples\test.doe`	Silent mode
`.\dough.cmd --debug examples\test.doe`	Debug mode

VS Code Extension

- Install from Marketplace: `code --install-extension aidanace3.dough-language`
- Or search in VS Code Extensions: `aidanace3.dough-language`

Features: syntax highlighting, keyword completion, snippet support, and inline error hints.

Install Runtime (CLI)

1. Open https://github.com/Aidanace3/Doe-Language/releases/latest
2. Download `dough-runtime-win-x64.zip`
3. Extract and run `Dough.exe yourfile.doe`
4. Optional: add the extracted folder to your PATH so `Dough.exe` works from any terminal.

VS Code `launch.json`

Add a `.vscode/launch.json` at the project root:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Dough: Run Current File",
      "type": "node-terminal",
      "request": "launch",
      "command": "powershell -NoProfile -ExecutionPolicy Bypass -File
          \"${workspaceFolder}/../Doe-Language/dough.ps1\" \"${file
         }\""
    },
    {
      "name": "Dough: Debug Current File",
      "type": "node-terminal",
      "request": "launch",
      "command": "powershell -NoProfile -ExecutionPolicy Bypass -File
          \"${workspaceFolder}/../Doe-Language/dough.ps1\" --debug \"
         ${file}\""
    }
  ]
}
```

Section 1: Basic Syntax

General Rules

- **Case-insensitive:** `Print`, `print`, and `PRINT` are identical.
- **Standalone booleans** are prefixed by `@`: e.g. `as (@true):`
- **Comments:** single-line `// comment` or multi-line `/* comment */`
- **Line endings** do not require semicolons except when closing a block definition.

Operators

Arithmetic Operators

Operator	Name	Notes
`+`	Add	
`-`	Subtract	
`**`	Multiply	`*` is reserved for points
`/`	Divide	
`^`	Exponentiate	
`%`	Percentage of	e.g. `50 % 200` → `100`
`%%`	Modulo (Remdiv)	remainder after division

Comparison & Logic Operators

Operator	Name	Notes
`==`	Equal	
`!`	Not	prefix: `!(x)`
`>`	Greater than	
`<`	Less than	
`=>`	Greater or equal	
`<=`	Less or equal	
`|`	Or	
`*|`	Xor	exclusive or
`&&`	And (Xand)	common and
`!&`	Nand	
`!|`	Nor	

Types

Type / Modifier	Description
`Str` / `String`	Text value
`Int`	Integer (no decimal)
`Flt`	Float (decimal)
`Bool`	Boolean (`@true` / `@false`)
`Arr[T]`	Typed array
`NoPoly`	Prevent implicit type coercion
`Const`	Immutable value
`Max(Arr)`	Upper bound on an array
`Min(Arr)`	Lower bound on an array

Variable declaration syntax:

```
[Modifier] [Type] name = value

// Examples:
Int count = 0
NoPoly Const Int maxSize = 100
Str greeting = "Hello"
```

I/O

Output

```
Print("Hello, world!")
Print("Count: " + count)       // concatenate with +
```

Input

```
Str name = Input("Enter your name: ")
Str pass = Input("Password: " -H)        // -H hides input
Str ans  = Input("Answer: " -W 10)       // -W 10 = 10-second timeout
```

`readln(n)` reads line #n directly from a file or buffer.

Conditions

If / Elif / Else

The `::` separator accepts `then` (run the block), `break` (exit the current scope), or a `FuncName()` to call a function before running the block.

```
if ( x > 10 )::then
{
    Print("big")
}
elif ( x == 10 )::then
{
    Print("exact")
}
else::
{
    Print("small")
}
```

Shorthand: When the body is a single expression, you may inline it on the same line as the condition:

```
if ( x == 0 )::then  x = 1
```

Calling a Function on Condition

```
NoPoly Const Int X = 5

def greet {
    Print("hi")
}

// greet() runs when X == 5, then the block also runs
if ( X == 5 )::greet()
{
    Return X >> this
}
```

Switch (IfCase)

```
IfCase(x)
{
    Case: x is 1:
    {
        Print("one")
    }
    Case: x is 2:
    {
        Print("two")
    }
    Default: x is outlier
    {
        Print("unknown: " + x)
    }
}
```

Functions

Functions are defined with `def`. Parameters are declared inside parentheses. Return values use `Return value >> *point`.

```
def add(Int a, Int b)
{
    Return (a + b) >> this
}

// Call it:
Int result = add(3, 4)
```

> **Note:** `Funcs` is deprecated. Use `def` for all function definitions.

Points

Points are named channels used for yielding and returning values. Think of them as lightweight, named slots that connect producers to consumers.

- Declared as `(*POINTNAME)`
- `value >> *Point` — sends a value to a point

- `yeild(var >> *Point)` — yields a value (non-blocking send)
- `awaitval` — triggers a block as soon as a value arrives at the point
- `exit(*Point)` — removes the point; call after a case/function completes
- `Store(val Asa name >> *Point)` — saves a named value to a point
- `request(x << *Point.name)` — retrieves a stored value from a point

Section 2: Examples & Syntax

Loops

Point Loop

A point loop uses `(*this)` to loop a fixed number of times:

```
{
(*);
{
    Print("looping")
    loop ( l >> this x10 )    // repeat 10 times
    return l >> this
}
exit(*this)
}
```

While Loop (as)

```
as (@true):
{
    // code
    if (@stopcondition)::break
}
```

For-Each Loop

```
each (x in [arr]) do:
{
    Print(x)
}
```

Counted For Loop

New in v0.8.0: `from...to` loop syntax for simple iteration over a numeric range.

```
from 0 to 10 as i:
{
    Print(i)
}

// With a step
from 0 to 100 step 5 as i:
{
    Print(i)
}
```

Arrays

Define and configure a typed array:

```
/( declare )\ name = Arr[Int]
/( set length )\ conf name.Length = 10
```

Uconf modifies object properties directly without re-assigning the object itself.
Array properties:

- `type` — element type (changeable unless `NoPoly`)
- `name` — identifier (constant)
- `length` — current capacity
- `lower` — lowest valid index (useful for offset constants)

Array Operations

```
/( define )\ nums = Arr[Int]
/( length  )\ conf nums.Length = 5

nums[0] = 10
nums[1] = 20

each (n in nums) do:
{
    Print(n)
}
```

Dictionaries

Standard Dictionary

```
dict UserProfile:
{
    Str name = "unnamed"
    Int age  = 0
    Bool active = @true
};
```

Access a field with dot notation: `UserProfile.name`

Locked (Single-Type) Dictionary

```
locked dict(Int) Scores:
{
    alice = 95
    bob   = 87
    carol = 91
};
```

> Use `locked dict` when all values share a type — it prevents accidental mixed-type inserts and enables tighter runtime checks.

Points — Full Examples

Basic Point (One-Shot)

```
(*Taking:) awaitval(x;)
{
    Print(x)
}

Int x = 5
x >> *Taking
// Output: 5
```

Continuous Listener

```
(*LogStream:) awaitval(msg;)
{
    Print("LOG: " + msg)
}

if ( logval == 1 )::then
    { "message 1" >> *LogStream }
elif ( logval == 2 )::then
    { "message 2" >> *LogStream }
else::break
```

Storing & Requesting Values via a Point

```
(*Config:)
{
    Store("dark" Asa theme >> *Config)
    Store(14     Asa fontSize >> *Config)
}

// Later, retrieve them:
Str t  = request(x << *Config.theme)
Int fs = request(x << *Config.fontSize)
Print("Theme: " + t + ", Size: " + fs)
```

Error Handling

New in v0.8.0: `try`/`catch`/`throw` for structured error handling.

```
try
{
    Int result = divide(10, 0)
}
catch (err)::then
{
    Print("Error: " + err)
}

def divide(Int a, Int b)
{
    if ( b == 0 )::then
        throw "Division by zero"
    Return (a / b) >> this
}
```

Pattern Matching (`match`)

New in v0.8.0: `match`/`when` provides expressive value-based branching — similar to `IfCase` but supports conditions and ranges.

```
match (score)
{
    when (score => 90)::then { Print("A") }
    when (score => 80)::then { Print("B") }
    when (score => 70)::then { Print("C") }
    else::                   { Print("F") }
}
```

Async Points & Concurrency

New in v0.8.0: `async`/`await` and `spawn` for concurrent execution using the Points model.

```
// Spawn a concurrent task; it runs alongside the main flow
spawn fetchData(*ResultPoint)

// The main thread continues here...
Print("Fetching in background...")

// Await blocks until *ResultPoint receives a value
await *ResultPoint as data
Print("Got: " + data)
```

`after` and `timeout` let you set a deadline on an `await`:

```
await *ResultPoint as data after 5000
{
    Print("Timed out after 5 seconds")
}
```

Imports & Modules

New in v0.8.0: Module import syntax for splitting code across files.

```
import utils
from math import add, multiply

Print(add(3, 4))
```

A module file (`math.doe`) simply contains `def` blocks at the top level — no special export keyword needed. All top-level definitions are importable by default.

Section 3: Quick Reference

Keyword Cheatsheet

YKeyword	YCategory	YPurpose
`def`	Function	Define a function
`return`	Function	Return a value via a point
`if / elif / else`	Control	Conditional branching
`match / when`	Control	Pattern matching
`IfCase / Case / Default`	Control	Switch statement
`as`	Loop	While loop
`each...in...do`	Loop	For-each loop
`from...to`	Loop	Counted range loop
`loop`	Loop	Repeat inside a point loop
`break`	Control	Exit current scope
`(*NAME)`	Points	Declare a point
`»`	Points	Send value to point
`yeild`	Points	Non-blocking send
`awaitval`	Points	React when value arrives
`await`	Async	Block until point receives
`spawn`	Async	Launch concurrent task
`after`	Async	Timeout for await
`exit(*P)`	Points	Remove a point
`Store`	Points	Save named value to point
`request`	Points	Read named value from point
`try / catch / throw`	Errors	Error handling
`dict`	Data	Dictionary definition
`locked dict`	Data	Single-type dictionary
`conf`	Data	Modify object properties
`NoPoly`	Type	No implicit coercion
`Const`	Type	Immutable value
`import / from`	Modules	Load external files

Common Patterns

Safe Division

```
def safeDivide(Int a, Int b)
{
    if ( b == 0 )::then
        throw "Cannot divide by zero"
    Return (a / b) >> this
}
```

Collecting Input into an Array

```
/( declare )\ inputs = Arr[Str]
/( length  )\ conf inputs.Length = 5

from 0 to 5 as i:
{
```

```
    inputs[i] = Input("Enter item " + i + ": ")
}
```

Event Bus with Points

```
(*Bus:) awaitval(event;)
{
    match (event)
    {
        when (event == "start")::then { Print("Starting...") }
        when (event == "stop")::then  { Print("Stopping...") }
        else::                        { Print("Unknown: " + event) }
    }
}

"start" >> *Bus
"stop"  >> *Bus
exit(*Bus)
```